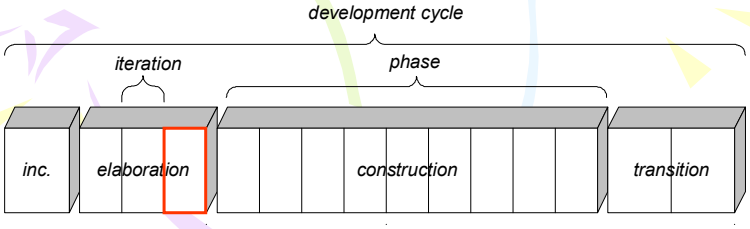
PART 5 Elaboration Iteration 3 – Intermediate topics



Chapter 27 Iteration 3 Intermediate Topics

Introduction

- Objective
  - Define the requirements for iteration-3
- NextGen POS Requirements in iteration-3 include
  - Provide failover to local services when the remote services cannot be accessed.
    - e.g., if the remote product database can't be accessed, use a local version with cached data.
  - Provide support for POS device handling, such as the cash drawer and coin dispenser.
  - Handle credit payment authorization.
  - Support for persistent objects.

3

Monopoly 1

- Monopoly requirements in iteration-3
  - There are now Lots, Railroads, and Utility squares. When a player lands on a Lot, Railroad or Utility square, the following logic applies…
  - If the Lot, Railroad or Utility square is not owned, the player who landed on the square may buy it. If they buy it, the price of the Lot, Railroad or Utility square is deducted from the player's money and the player becomes its owner.
    - The price is set when the game starts, but is arbitrary — for example, the official Monopoly prices may be used.
  - If the Lot, Railroad or Utility square is owned by the player that landed on it, nothing happens
  - If the Lot, Railroad or Utility square is owner by a player other than the player that landed on it, the player that landed on the square must pay its owner rent. The rent calculations are:
    - Lot rent is (index position) dollars; e.g., if position 5, then $5.
    - Railroad rent is 25 dollars times the number of Railroads owned by the owner; e.g., if own 3 Railroads, then $75.
    - Utilities rent is 4 times the number shown on the dice when the player lands on the square (do not roll again)

4

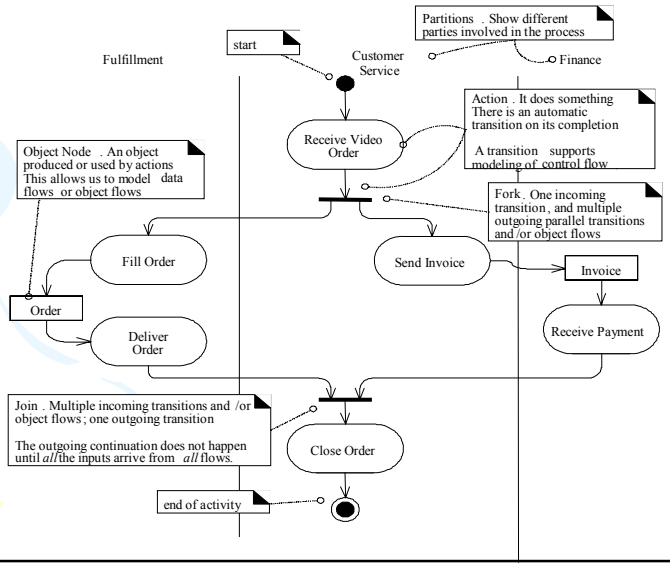
Chapter 28 UML Activity Diagrams and Modeling

Activity Diagram 1

- A UML activity diagram shows sequential and parallel activities in a process.
  - They are useful for modeling business processes, workflows, data flows, complex algorithms, and use cases
  - This diagram shows a sequence of actions, some of which may be parallel
  - Once an action is finished, there is an automatic outgoing transition
  - The diagram can show both control flow and data flow
- Next slide illustrate
  - An action, partition, fork, join, and object node.

6

Activity Diagram 2

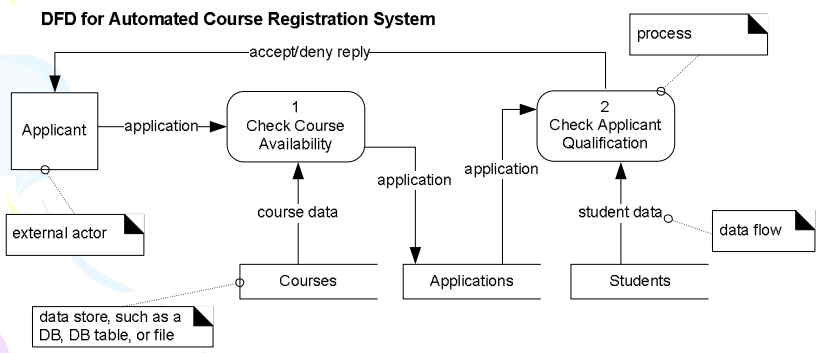


How to Apply Activity Diagrams? 1

- **Business Process Modeling**
  - Activity diagrams can be used to understand the complex business processes by visualizing them.
  - The partitions are useful to see the multiple parties and parallel actions involved in the business process
- **Data Flow Modeling**
  - Starting in the 1970s, data flow diagrams (DFD) became a popular way to visualize the major steps and data involved in software system processes
  - The information modeled in a DFD is useful, both for documentation and discovery
    - UML activity diagrams can satisfy the same goals
  - Notice that in addition to **object nodes** being useful to show data flow, the UML **datastore node** is applicable

How to Apply Activity Diagrams?

2

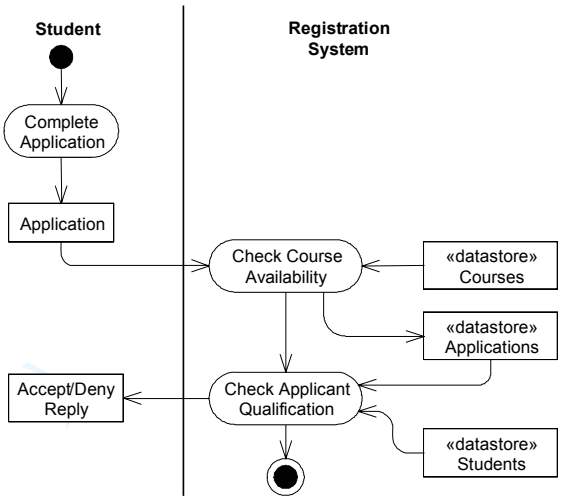


Classic DFD in Gane-Sarson notation

9

How to Apply Activity Diagrams?

3



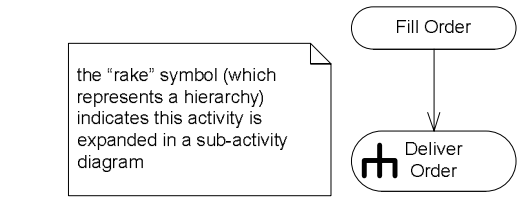
10

Applying activity diagram notation to show a data flow model

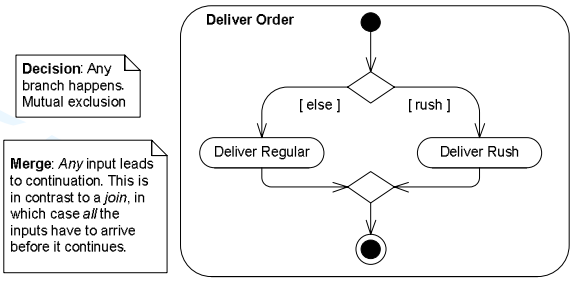
How to Apply Activity Diagrams? 4

- **Concurrent Programming and Parallel Algorithm Modeling**
  - Parallel algorithms in concurrent programming problems involve multiple partitions, and fork and join behavior
    - For example, such algorithms are used in 3D simulations with finite element or finite difference modeling, and are applied to oil reservoir modeling, materials stress analysis, and weather modeling
    - The overall physical space is subdivided into large blocks, and many parallel threads (or processes) execute, one for each sub-block
  - In these cases the UML activity diagram partitions can be used to represent different operating system threads or processes.
  - The object nodes can be used to model the shared objects and data.
  - Forking can be used to model the creation and parallel execution of multiple threads or processes, one per partition

Expanded activity to Another Diagram

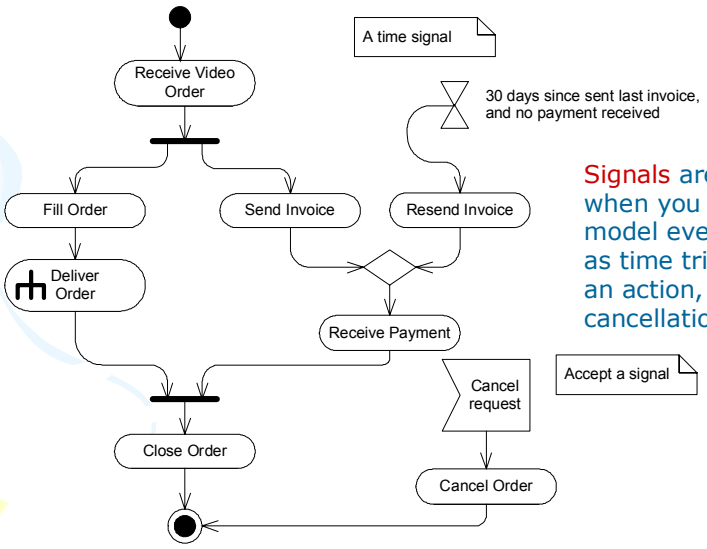


An activity will be expanded in another diagram



The expansion of an activity

Model Events as Action

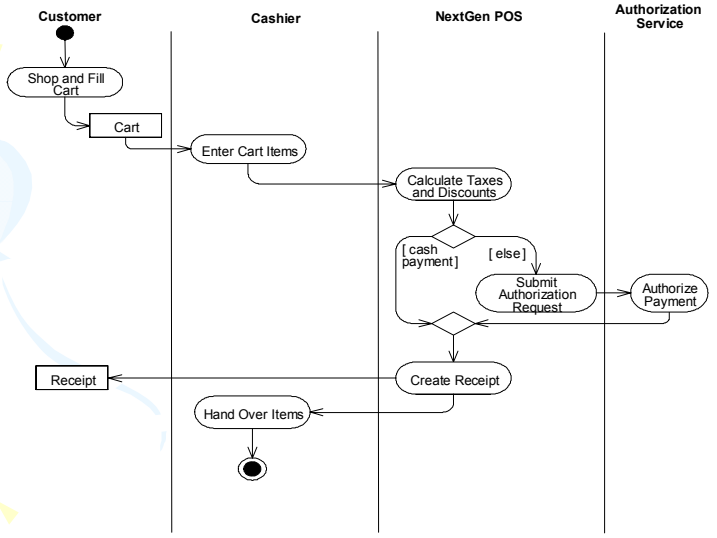


Signals are useful when you need to model events such as time triggering an action, or a cancellation request.

Guidelines for Activity Modeling

- Guidelines
  - This technique proves most valuable for very complex processes, usually involving many parties.
  - If modeling a business process, take advantage of the "rake" notation and sub-activity diagrams.
    - "level 0" diagram keep all the actions at a very high level of abstraction. Expand the details in sub-diagrams at the level 1, and perhaps even more at the level 2.
  - Strive to make the level of abstraction of action nodes roughly equal within a diagram.
    - Bad example: suppose in a "level 0" diagram there is an action node labeled "Deliver Order." And, a second action node "Calculate Tax." Those are very different levels of abstraction.

NextGen POS Activity Diagram



Modeling the Process Sale use case with an activity diagram

15

Activity Diagrams in the UP

- UP disciplines - Business Modeling
  - Its purpose is to understand and communicate the structure and the dynamics of the organization in which a system is to be deployed.
  - A key artifact of the business modeling discipline is the Business Object Model (a superset of the UP domain model), which essentially visualizes how a business works, using UML class, sequence, and activity diagrams.
  - Thus, activity diagrams are especially applicable within the Business Modeling discipline of the UP

16

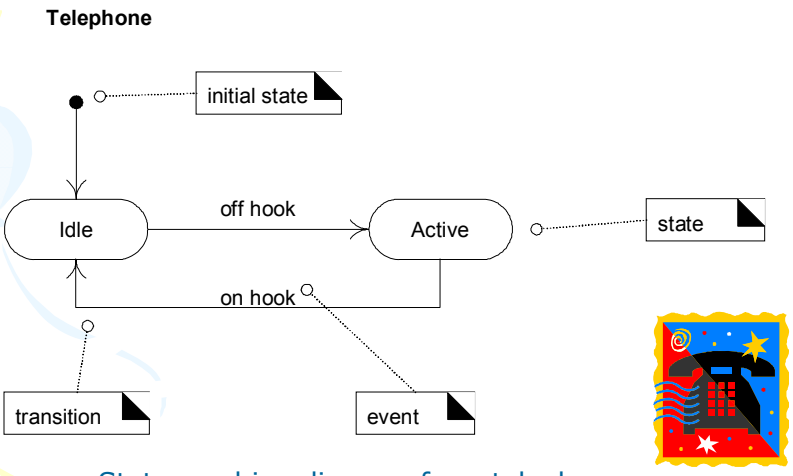
Chapter 29 UML State Machine Diagrams and Modeling

State Machine Diagram 1

- A UML state machine diagram
  - Shows the lifecycle of an object: what events it experiences, its transitions, and the states it is in between these events.
  - It need not illustrate every possible event.
  - Transitions are shown as arrows, labeled with their event.
  - States are shown in rounded rectangles.
  - initial pseudo-state automatically transitions to another state when the instance is created.

18

State Machine Diagram 2



State machine diagram for a telephone

Events, States, and Transitions

- An **event** is a significant or noteworthy occurrence.
  - A telephone receiver is taken off the hook.
- A **state** is the condition of an object at a moment in time between events.
  - A telephone is in the state of being "idle" after the receiver is placed on the hook and until it is taken off the hook.
- A **transition** is a relationship between two states
  - when an event occurs, the object moves from the prior state to the subsequent state.
  - e.g. When the event "off hook" occurs, transition the telephone from the "idle" to "active" state.

Apply State Machine Diagrams 1

- **State-independent (or modeless)**
  - The object always responds the same way to an event.
- **State-independent objects**
  - For all the events of interest, the object always reacts the same way
- **State-dependent objects**
  - React differently to events depending on their state or mode.
- Guideline
  - Consider state machines for state-dependent objects with complex behavior.
    - E.g., the phone's reaction to pushing a particular button (generating an event) depends on the current mode of the phone — off hook, engaged, in a configuration subsystem, and so forth

21

Apply State Machine Diagrams 2

- Guideline
  - In general, business information systems have few complex state-dependent classes. It is seldom helpful to apply state machine modeling.
  - Process control, device control, protocol handlers, and telecommunication domains have many state-dependent objects.
- Broadly, state machines are applied in two ways:
  - To model the behavior of a complex reactive object in response to events.
  - To model legal sequences of operations — protocol or language specifications.
    - This approach may be considered a specialization of #1, if the "object" is a language, protocol, or process. A formal grammar for a context-free language is a kind of state machine

22

23

Apply State Machine Diagrams 3

- Complex Reactive Objects
  - Physical Devices controlled by software
    - Phone, car, microwave oven. They have complex and rich reactions to events, and the reaction depends upon their current mode.
  - Transactions and related Business Objects
    - How does a business object (a sale, order, payment) react to an event? For example, what should happen to an Order if a cancel event occurs?
    - Understanding all the events and states that a *Package* can go through in the shipping business can help with design, validation, and process improvement.
  - Role Mutators (The objects change their role)
    - A Person changing roles from being a civilian to a veteran. Each role is represented by a state.

24

Apply State Machine Diagrams 4

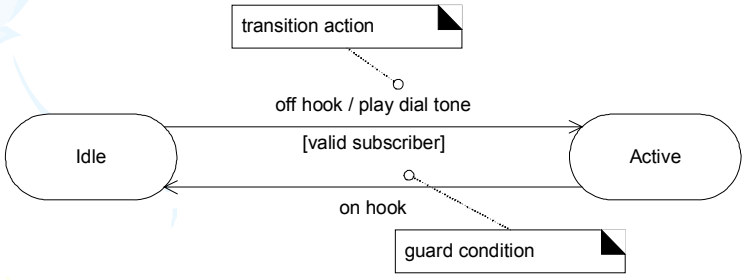
- Protocols and Legal Sequences
  - Communication Protocols
    - TCP, and new protocols can be easily and clearly understood with a state machine diagram.
      - The diagram illustrates when operations are legal.
      - For example, a TCP "close" request should be ignored if the protocol handler is already in the "closed" state.
  - UI Page/Window Flow or Navigation
    - When doing UI modeling, it can be useful to understand the legal sequence between Web pages or windows; this is often complex.
  - UI Flow Controllers or Sessions
    - This is related to UI navigation modeling, but specifically focused on the server-side object that controls page flow.
    - These are usually server-side objects representing an ongoing session or conversations with a client.
      - e.g., a Web application that remembers the state of the session with a Web client and controls the transitions to new Web pages, or the modified display of the current Web page, based upon the state of the session and the next operation that is received.

Apply State Machine Diagrams 5

- Protocols and Legal Sequences (cont'd)
  - Use Case System Operations
    - System operations for Process Sale: makeNewSale, enterItem, etc. should arrive in a legal order;
      - e.g., endSale should come after one or more enterItem operations.
    - If the order is complex, a state machine can model this, treating the use case itself as an object.
  - Individual UI Window Event Handling
    - Understanding the events and legal sequences for one window or form.
    - e.g., the Edit-Paste action is only valid if there is something in the "clipboard" to paste.

Transition Actions and Guards

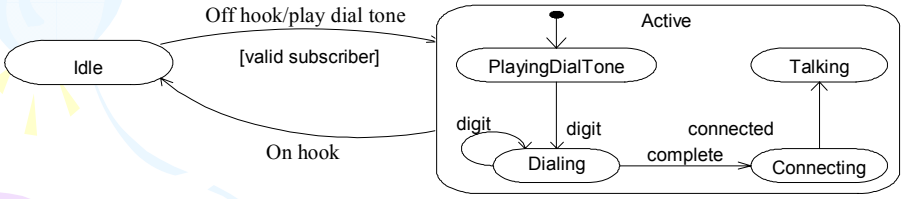
- A transition can cause an action to fire.
  - This may be implemented as the invocation of a method of the class of the state machine diagram.
- A transition may also have a conditional guard or boolean test.
  - The transition only occurs if the test passes.



Transition action and guard notation

Nested States

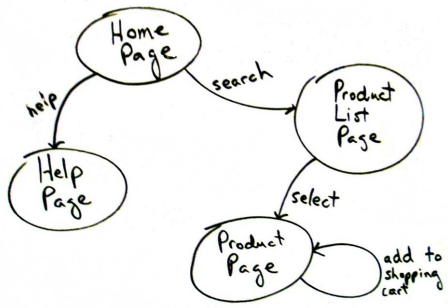
- A state allows nesting to contain substates
  - A substate inherits the transitions of its superstate (the enclosing state)
  - It leads to succinct state machine diagrams. Substates may be graphically shown by nesting them in a superstate box



  - e.g., when a transition to the Active state occurs, creation and transition into the PlayingDialTone substate occurs. No matter what substate the object is in, if the on hook event related to the Active superstate occurs, a transition to the Idle state occurs.

UI Navigation Modeling with State Machines

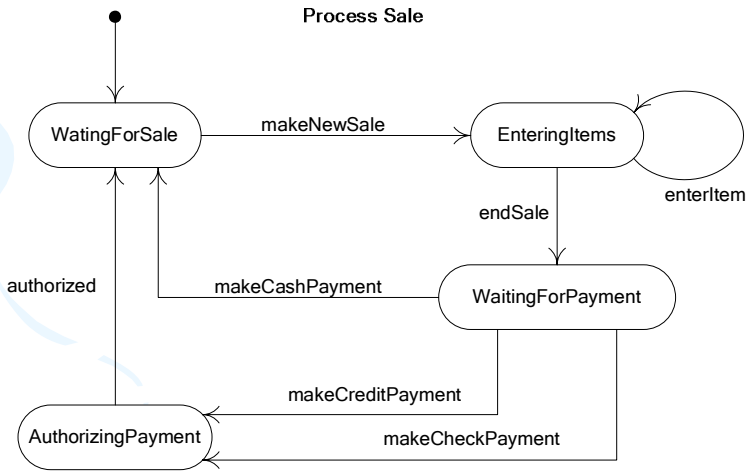
- Some (Web) UI applications have complex page flows.
  - State machines are a great way to model page flows, during creative design.
  - States: pages; Events: events that cause transfer from one page to another (e.g., button click).



29

Applying a state machine to Web page navigation modeling

Example: NextGen Use Case State Machine Diagram



30

A sample state machine for legal sequence of use case operations

State Machine Diagrams in the UP

- There is not one model in the UP called the "state model."
  - Any element in any model (Design Model, Domain Model, Business Object Model) may have a state machine to better understand or communicate its dynamic behavior
  - e.g., a state machine associated with the Sale design class of the Design Model is itself part of the Design Model

31

Chapter 30 Relating Use Cases

Relating Use Cases

- Objective:
  - Relate uses cases with *include* and *extend* associations, in both text and diagram formats.
- Use cases can be related to each other.
  - For example, a subfunction use case such as Handle Credit Payment may be part of several regular use cases, such as Process Sale and Process Rental.
- Organizing use cases into relationships has no impact on the behavior or requirements of the system.
  - Rather, it is simply an organization mechanism to improve communication and comprehension of the use cases, reduce duplication of text, and improve management of the use case documents

33

Guideline: Avoid Agonizing Over Use Case Relationships

- In some organizations working with use cases
  - Way too much unproductive time is spent debating how to relate use cases in a use case diagram, rather than the important use case work: **writing text**
  - It actually reflects a deeper problem in analysis-oriented work on software projects: Too much time wasted on low-value analysis and modeling.
    - If you think you have to "get it right" at the start, everything becomes bogged down in analysis paralysis
  - If a team starts off use-case modeling by spending hours (or worse, days) discussing a use case diagram and use case relationships rather than quickly focusing on writing the key use case text, relative effort was misplaced
- In UP and other iterative methods
  - The organization of use cases into relationships can iteratively evolve in small steps over the elaboration phase

34

The include Relationship 1

- The include relationship is the most common and important relationship.
  - Some partial behavior across several use cases. (most common)
    - E.g., paying by credit occurs in several use cases, including Process Sale, Process Rental, Contribute to Lay-away Plan.
    - Rather than duplicate this text, it is desirable to separate it into its own subfunction use case, and indicate its inclusion.
    - This is simply refactoring and linking text to avoid duplication
- **Guideline:**
  - Use include when you are repeating yourself in two or more separate use cases and you want to avoid repetition
  - To decompose an overwhelmingly long use case into subunits to improve comprehension.

35

The include Relationship 2

UC1: Process Sale

Main Success Scenario:
1. Customer arrives at a POS checkout with goods and/or services to purchase
....
7. Customer pays and System handles payment....

Extensions:
7b. Paying by credit: Include Handle Credit Payment.
7c. Paying by check: Include Handle Check Payment....

UC7: Process Rental

Extensions:
6b. Paying by credit: Include Handle Credit Payment.

Underline indicates an included use case

UC12: Handle Credit Payment

Level: Subfunction

Main Success Scenario:
1. Customer enters their credit account information.
2. System sends payment authorization request to an external Payment Authorization Service System, and requests payment approval.
3. System receives payment approval and signals approval to Cashier.

Extensions:
2a. System detects failure to collaborate with external system:
System signals error to Cashier.
Cashier asks Customer for alternate payment.

36

The include Relationship 3

- Using include with Asynchronous Event Handling
  - When a user is able to, at any time, select or branch to a particular window, function, or Web page, or within a range of steps.
  - Basic notation is to use a*, b*, ... style labels in the Extensions section to imply an extension or event that can happen at any time.
  - A minor variation is a range label, such as 3-9, to be used when the asynchronous event can occur within a relatively large range of the use case steps, but not all

UC1: Process FooBars

Main Success Scenario:

...

Extensions:

a*. At any time, Customer selects to edit personal information: **Edit Personal Information**.

b*. At any time, Customer selects printing help: **Present Printing Help**.

2-11. Customer cancels: **Cancel Transaction Confirmation**

Underline indicates an included use case

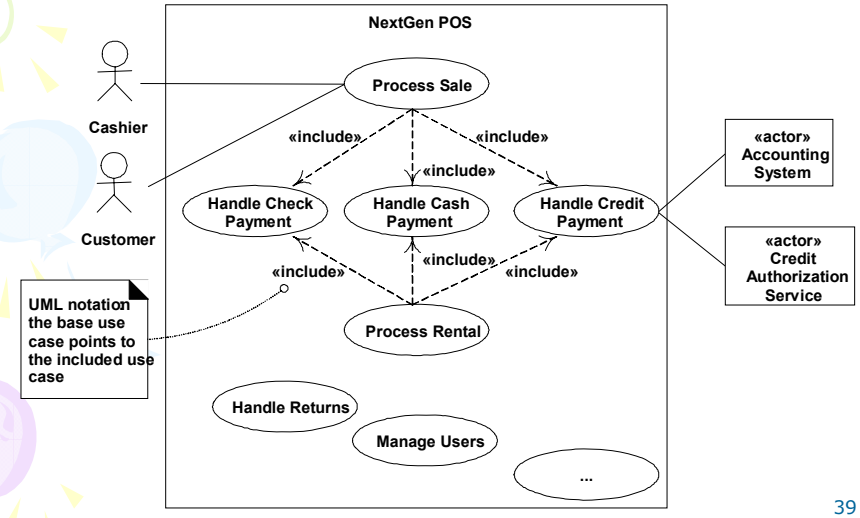
37

The include Relationship 4

- There are other relationships: Extend and Generalization
  - But Cockburn, an expert use-case modeler, advises to prefer the include relationship over extend and generalization.
  - As a first rule of thumb
    - Always use the include relationship between use cases.
    - People who follow this rule report they and their readers have less confusion with their writing than people who mix include with extend and generalizes [Cockburn01].

38

The include Relationship 5



Use case include relationship in the Use-Case Model

Concrete/Abstract Use Cases

- Concrete use case
  - Initiated by an actor and performs the entire behavior desired by the actor.
  - Are the elementary business process use cases
  - Process Sale is a concrete use case.
- Abstract use case
  - Never instantiated by itself; it is a subfunction use case that is part of another use case.
  - Handle Credit Payment is abstract; it doesn't stand on its own, but is always part of another story, such as Process Sale.

Base/Addition Use Cases

- Base use case
  - A use case that includes another use case, or is extended or specialized by another use case.
  - Process Sale is a base use case with respect to the included Handle Credit Payment.
- Addition use case
  - The use case that is an inclusion, extension, or specialization.
  - Handle Credit Payment is the addition use case in the include relationship to Process Sale.
- Addition use cases are usually abstract. Base use cases are usually concrete.

41

The extend Relationship 1

- The extend relationship
  - Suppose a use case's text should not be modified
    - Continually modifying the use case with myriad new extensions and conditional steps is a maintenance headache
    - It has been baselined as a stable artifact, and can't be touched.
    - How to append to the use case?
  - The extend relationship allow to create an extending or addition use case, and within it, describe where and under what condition it extends the behavior of some base use case.

42

The extend Relationship 1

Point of extension

UC1: Process Sale (the base use case)

Extension Points: VIP Customer, step 1. Payment, step 7.

Main Success Scenario:

1. Customer arrives at a POS checkout with goods and/or services to purchase
....
7. Customer pays and System handles payment....

UC15: Handle Gift Certificate Payment (the extending use case)

Trigger: Customer wants to pay with gift certificate.

Extension Points: Payment in Process Sale.

Level: Subfunction

Main Success Scenario:

1. Customer gives gift certificate to Cashier.
2. Cashier enters gift certificate ID.

43

The extend Relationship 2

- Note the use of an extension point
  - The extending use case is triggered by some condition.
  - Extension points are labels in the base use case which the extending use case references as the point of extension
    - So that the step numbering of the base use case can change without affecting the extending use case
  - The extension point may simply "At any point in use case X,"
    - Common in systems with many asynchronous events, such as a word processor ("do a spell check now," "do a thesaurus lookup now"), or reactive control systems.
- A signature quality of the extend relationship is that the base use case (Process Sale) has no reference to the extending use case (Handle Gift Certificate Payment)
  - Does not define or control the conditions under which the extensions trigger

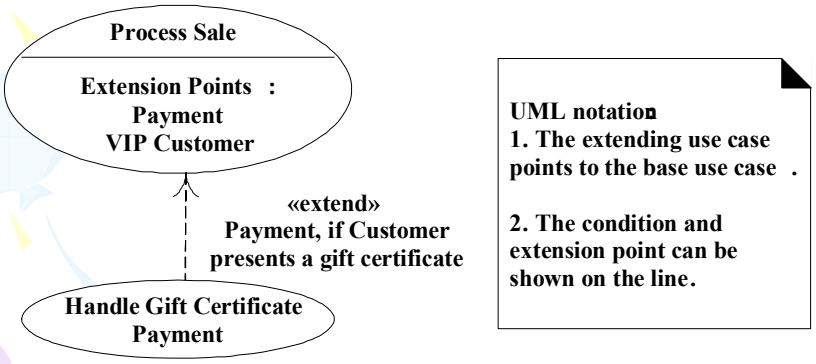
44

The extend Relationship 3

- The Handle Gift Certificate Payment addition use case could alternatively have been referenced within Process Sale with an include relationship, as with Handle Credit Payment
  - The extend relationship is used because we assume that the Process Sale use case was not to be modified
- Note that this gift certificate scenario could simply have been recorded by adding it as an extension in the Extensions section of Process Sale
  - This avoids both the include and extend relationships, and the creation of a separate subfunction use case
  - Just updating the Extensions section is usually the preferred solution, rather than creating complex use case relationships.
- Practical motivation of using the extend technique is when it is undesirable for some reason to modify the base use case.

45

The extend Relationship 4



The extend relationship

46

The generalize Relationship

- Generalize relationship
  - Outside the scope of this book.
  - Use case experts have been successfully doing use case work without generalize relationship
  - The generalize relationship adds another level of complexity to use cases.
  - There is not yet agreement by practitioners on the best-practice guidelines of how to get value from this idea.
  - A common observation by use case consultants is that complications result and unproductive time is spent on the addition of many use case relationships.

Chapter 31
More SSDs and Contracts

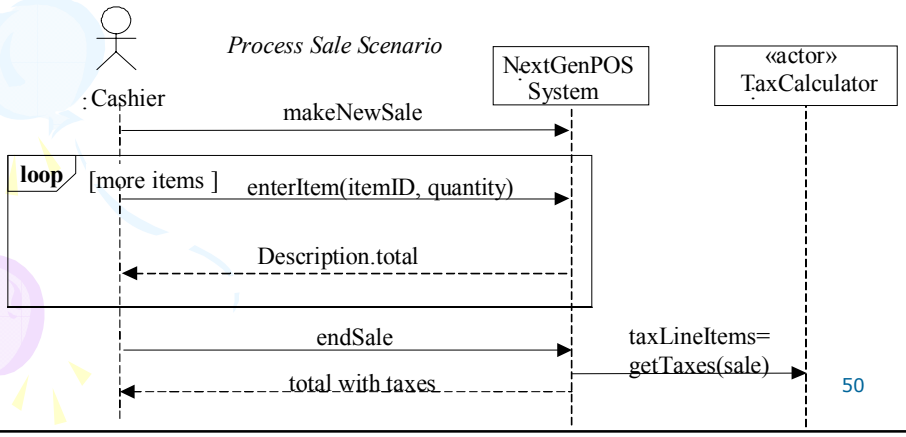
NextGen POS 1

- **Objective**
  - Define SSDs and operation contracts for the current iteration (iteration 3)
- **New System Sequence Diagrams**
  - In the current iteration, the new payment handling requirements involve new collaborations with external systems.
  - It is useful to illustrate the new system events in SSDs to clarify:
    - New system operations to support
    - Calls to other systems, and the responses to expect from these calls

49

NextGen POS 2

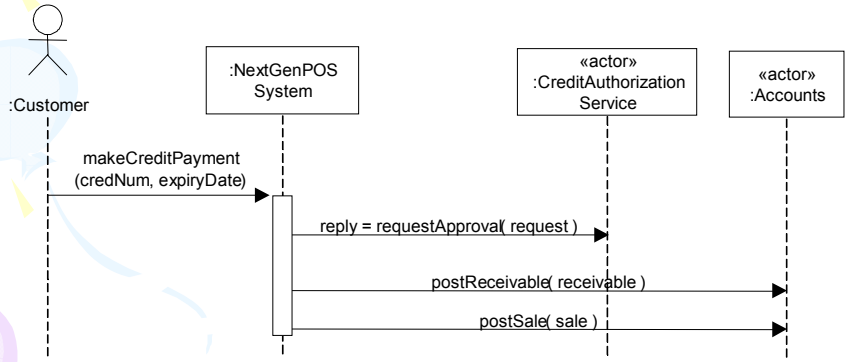
- **Common Beginning of Process Sale Scenario**
  - A basic scenario includes makeNewSale, enterItem, and endSale system events - common regardless of the payment method



50

NextGen POS 3

This **credit payment scenario** SSD starts after the common beginning

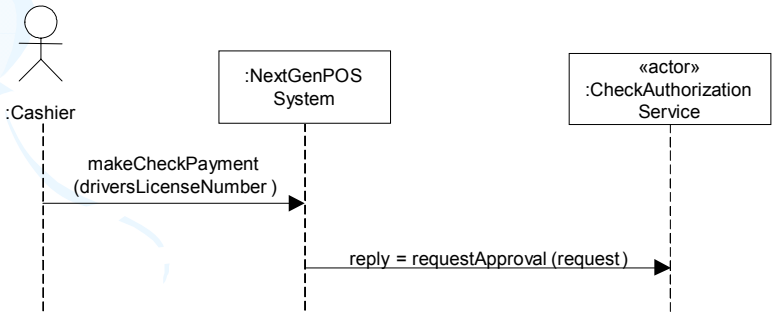


Credit payment SSD

51

NextGen POS 5

- This **check payment scenario** SSD starts after the common beginning
  - According to the use case, the cashier must enter the driver's license number for validation



Check payment SSD

52

NextGen POS 4

- In both cases of credit and check payments
  - This iteration assumes the payment is exactly equal to the sale total, and a different amount does not have be an input parameter.
- For credit payment
  - The call to the external creditauthorization service is modeled as a regular synchronous message with a return value.
    - This is an abstraction that could be implemented with a soap request over secure https, or any remote communication mechanism.
  - The makecreditpayment system operation and the use case assume
    - The credit information (credit account number and expiry date) of the customer is coming from a credit card (reader).
  - When a credit authorization service approves a credit payment, it owes the store for the payment; thus, a receivables entry needs to be added to the accounts receivable system.

53

NextGen POS 6

- In this iteration, the new system operations must handle are:
  - makeCreditPayment
  - makeCheckPayment
- In the first iteration, the system operation for the cash payment was simply makePayment.
  - makePayment → makeCashPayment.
- To review, system operation contracts are an optional requirements artifact (part of the Use-Case Model)
  - Usually, the use case text is itself sufficient, and these contracts aren't useful.
  - But on occasion they bring value by their precise and detailed approach to identifying what happens when a complex operation is invoked on the system, in terms of state changes to objects defined in the Domain Model.

54

NextGen POS 7

Contract CO5: makeCreditPayment

Operation:makeCreditPayment(creditAccountNumber, expiryDate)

Cross References:Use Cases: Process Sale

Preconditions: An underway sale exists and all items have been entered

Postconditions:

- a CreditPayment pmt was created
- pmt was associated with the current Sale sale
- a CreditCard cc was created; cc.number = creditAccountNumber, cc.expiryDate = expiryDate
- cc was associated with pmt
- a CreditPaymentRequest cpr was created
- pmt was associated with cpr
- a ReceivableEntry re was created
- re was associated with the external AccountsReceivable
- sale was associated with the Store as a completed sale

- Note:
  - Postcondition indicating the association of a new receivable entry in accounts receivable.
    - During testing, it is clear from this post-condition that the accounts receivable system should be tested for the presence of a new receivable entry.

NextGen POS 8

Contract CO6: makeCheckPayment

Operation:makeCheckPayment(driversLicenceNumber)

Cross References:Use Cases: Process SalePreconditions:An underway sale exists and all items have been entered.

Postconditions:

- a CheckPayment pmt was created
- pmt was associated with the current Sale sale
- a DriversLicense dl was created; dl.number = driversLicenseNumber
- dl was associated with pmt
- a CheckPaymentRequest cpr was created.
- pmt was associated with cpr
- sale was associated with the Store as a completed sale

Chapter 32 Domain Model Refinement

Objective

- Objective
  - Refine the domain mode with generalizations, specializations, association classes, time intervals, composition, and packages
  - Identify when showing a subclass is worthwhile

58

Introduction

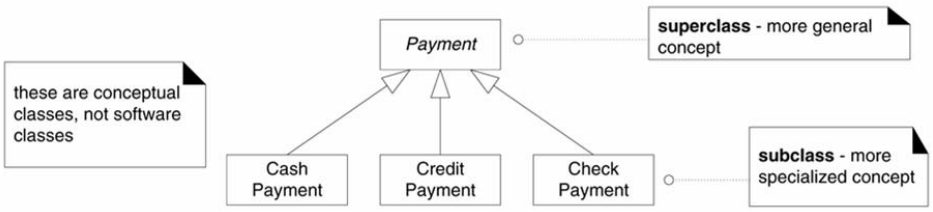
- Generalization and specialization
  - are fundamental concepts in domain modeling that supports an economy of expression
- Conceptual class hierarchies
  - are often the basis of inspiration for software class hierarchies that exploit inheritance and reduce duplication of code
- Association classes
  - capture information about an association itself
- Time intervals
  - capture the important concept that some business objects are valid for a limited time
- Packages
  - are a way to organize large domain models into smaller units.

59

60

Generalization

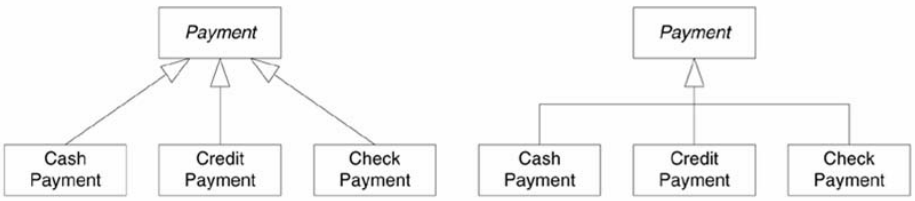
- The concepts CashPayment, CreditPayment, and CheckPayment are very similar
  - It is possible to organize them into a generalization-specialization class hierarchy
    - Superclass (*Payment*) : more general concept
    - Subclasses: more specialized concepts



61

Generalization

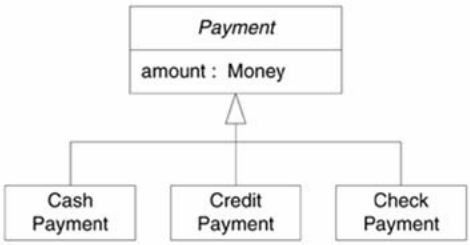
- Guideline: Identify domain superclasses and subclasses relevant to the current iteration, and illustrate them in the domain model.
- UML
  - The generalization relationship is indicated with a large hollow triangle.



62

Defining conceptual superclasses and subclasses

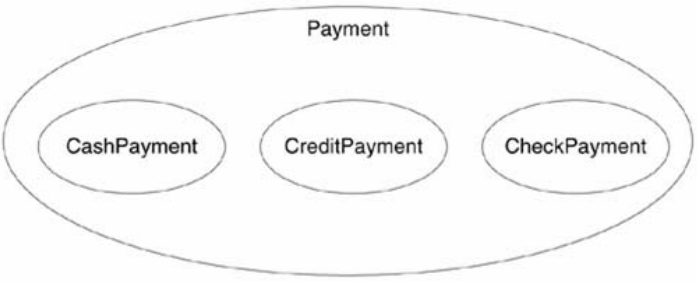
- Definition:
  - A conceptual superclass definition is more general or encompassing than a subclass definition
  - For example
    - Assume Payment represents the transaction of transferring money
    - A CreditPayment is a transfer of money via a credit institution which needs to be authorized.



63

Defining conceptual superclasses and subclasses

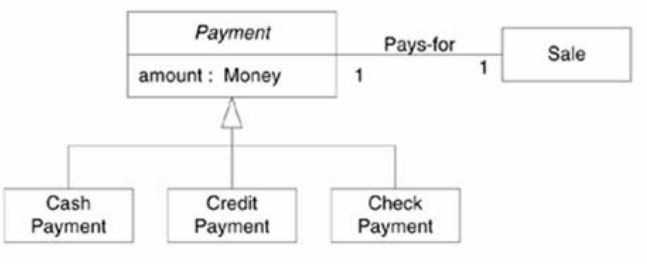
- Definition:
  - All members of a conceptual subclasses set are members of their superclass set.



64

Defining conceptual superclasses and subclasses

- Guideline: 100% rule
  - 100% of the conceptual superclass's definition should be applicable to the subclass, i.e., conform to 100% of the superclass's
    - Attributes
    - Associations

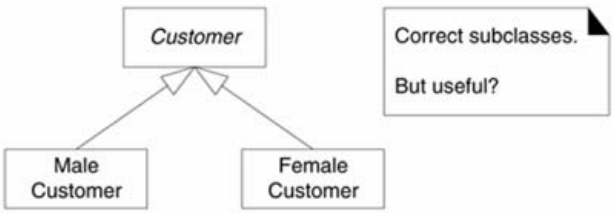


Defining conceptual superclasses and subclasses

- Guideline: Is-a rule
  - All members of a subclass set must be members of the superclass set
  - In natural language, this can usually be informally tested by forming the statement: Subclass is a Superclass
- Guideline:
  - A potential subclass should conform to
    - 100% rule (definition conformance)
    - Is-a rule (set membership conformance)

When to Define a Conceptual Subclass?

- In POS domain
  - Customer may be correctly partitioned into MaleCustomer and FemaleCustomer.
  - Is it useful?



67

When to Define a Conceptual Subclass?

- Guideline: Create a conceptual subclass of a superclass when
  - The subclass has additional attribute of interest
  - The subclass has additional associations of interest
  - The subclass concept is operated on, handled, reacted to, or manipulated differently than the superclass or other subclasses, in ways that are of interest
  - The subclass concept represents an animate thing (for example, animal robot) that behaves differently than the superclass or other subclasses in ways that are of interest.

68

When to Define a Conceptual Subclass?

Conceptual Subclass Motivation	Examples
The subclass has additional attributes of interest.	Payments not applicable. Library Book, subclass of LoanableResource, has an ISBN attribute.
The subclass has additional associations of interest.	Payments CreditPayment, subclass of Payment, is associated with a CreditCard. Library Video, subclass of LoanableResource, is associated with Director.
The subclass concept is operated upon, handled, reacted to, or manipulated differently than the superclass or other subclasses, in ways that are of interest.	Payments CreditPayment, subclass of Payment, is handled differently than other kinds of payments in how it is authorized. Library Software, subclass of LoanableResource, requires a deposit before it may be loaned.
The subclass concept represents an animate thing (for example, animal, robot) that behaves differently than the superclass or other subclasses, in ways that are of interest.	Payments not applicable. Library not applicable. Market Research MaleHuman, subclass of Human, behaves differently than FemaleHuman with respect to shopping habits.

69

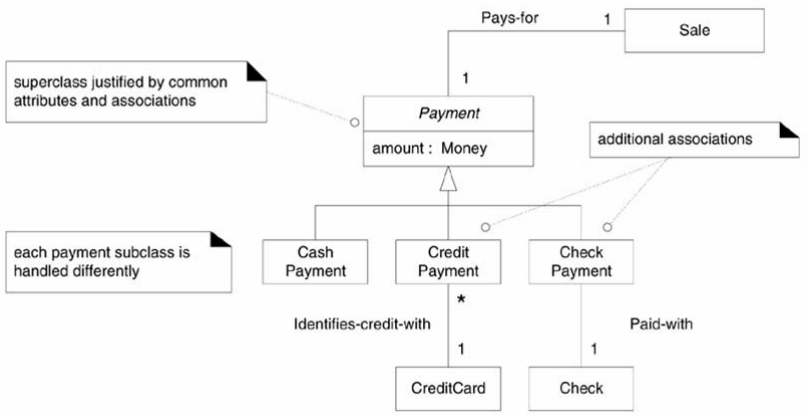
When to Define a Conceptual Subclass?

- Guideline: Create a superclass in a generalization relationship to subclasses when
  - The potential conceptual subclasses represent variations of a similar concept.
  - The subclasses will conform to the 100% and Is-a rules.
  - All subclasses have the same attribute that can be factored out and expressed in the superclass.
  - All subclasses have the same association that can be factored out and related to the superclass.

70

When to Define a Conceptual Subclass?

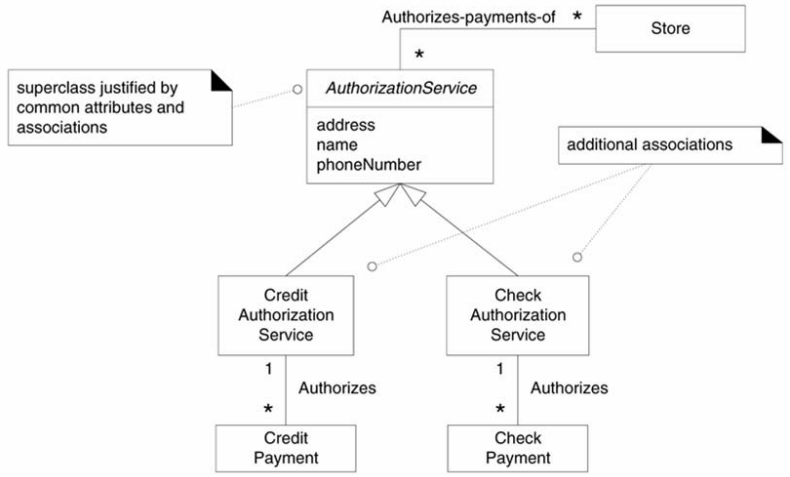
- Justifying Payment subclasses



. 1

When to Define a Conceptual Subclass?

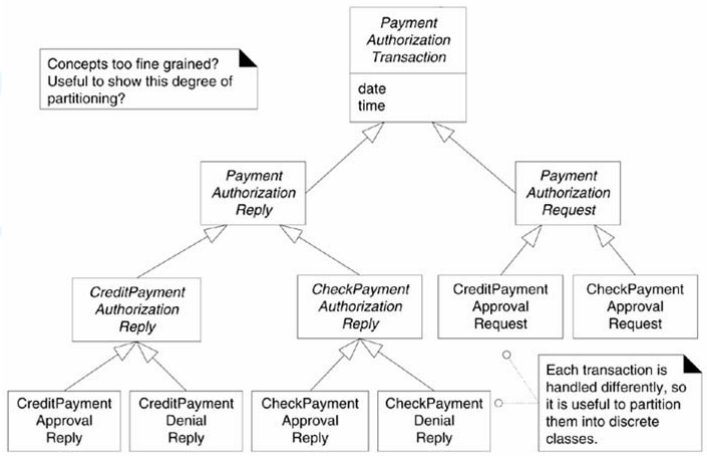
- Justifying AuthorizationService hierarchy



72

When to Define a Conceptual Subclass?

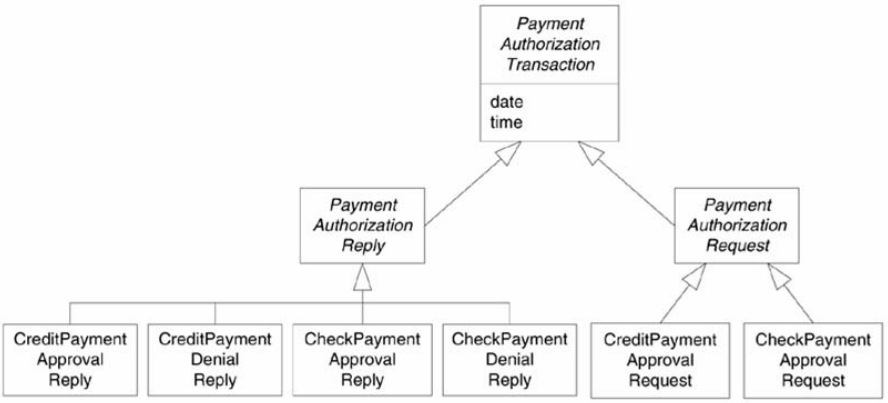
- One possible class hierarchy for external service transactions



73

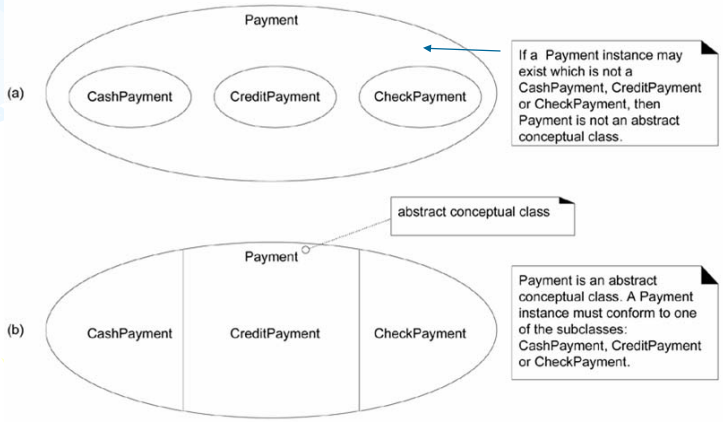
When to Define a Conceptual Subclass?

- An alternate transaction class hierarchy.



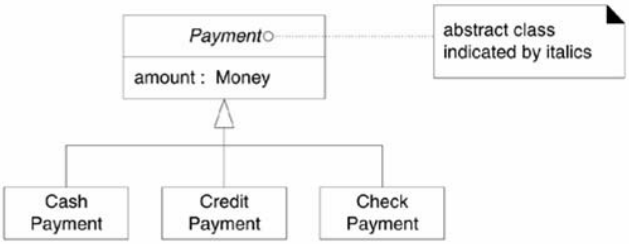
Abstract Conceptual Classes

- Definition:
  - If every member of a class C must also be a member of a subclass, then class C is called an abstract conceptual class.



Abstract Conceptual Classes

- UML provides a notation to indicate abstract classes
  - The class name is italicized
- Guideline: Identify abstract classes and illustrate them with an italicized name in the Domain model, our use the {abstract} keyword.



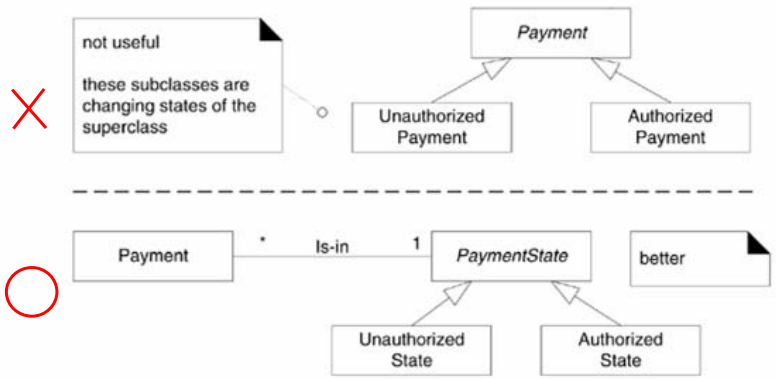
77

Modeling Changing States

- Modeling authorized/unauthorized payments (next page)
  - Suppose a payment can either be in an unauthorized or authorized state
  - Suppose it is meaningful to show this in the domain model
    - It may not really be meaningful, but assume for the discussion
- Guideline: Do not model the states of a concept X as subclasses of X. Rather, either
  - Define a state hierarchy and associate the states with X, or
  - Ignore showing the states of a concept in the domain model; show the states in state diagrams instead

78

Modeling Changing States



79

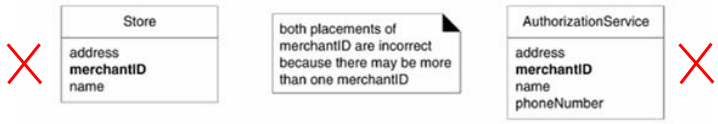
Class Hierarchies and Inheritance in Software

- This discussion of conceptual class hierarchies has not mentioned inheritance
  - because the discussion is focused on a domain model conceptual perspective, not software objects.
- In an object-oriented language
  - Inheritance is a software mechanism to make superclass things applicable to subclasses.
  - Inheritance has no real part to play in the discussion of the domain model, although it most definitely does when we transition to the design and implementation view.
- The conceptual class hierarchies generated here may or may not be reflected in the Design Model.
  - For example, the hierarchy of authorization service transaction classes may be collapsed or expanded into alternate software class hierarchies

80

Association Classes

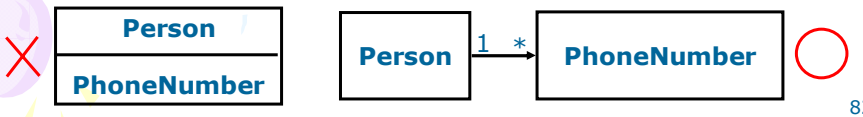
- The following domain requirements set the stage for association classes
  - Authorization services assign a merchant ID to each store for identification during communications
  - A payment authorization request from the store to an authorization service needs the merchant ID that identifies the store to the service
  - Furthermore, a store has a different merchant ID for each service
- Where in the UP Domain Model should the merchant ID attribute reside?
  - Placing merchant ID in Store is incorrect because a Store can have more than one value for merchant ID. The same for Authorization Service.



81

Association Classes

- Guideline: In a domain model, if a class C can simultaneously have many values for the same kind of attribute A, do not place attribute A in C. Place attribute A in another class that is associated with C
  - For example: A Person may have phone numbers. Place phone numbers in another class, such as PhoneNumber or ContactInformation, and associate many of these to Person



82

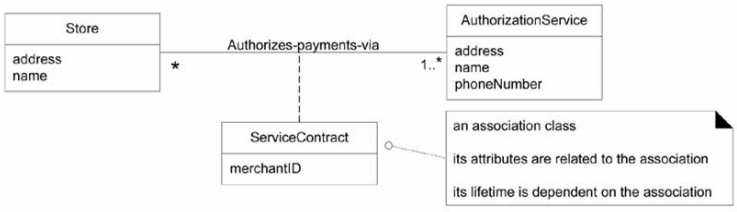
Association Classes

- More appropriate, but …
  - Merchant ID may be though of as an attribute related to the association between Store and AuthorizationService
  - This leads to the notion of an association class

83

Association Classes

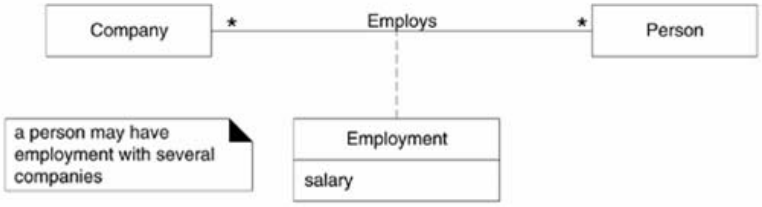
- Better
  - ServiceContract may be modeled as an association class related to the association between Store and Authorization service.
  - Association class: we can add features to the association itself.
  - In UML, this is illustrated with a dashed line from the association to the association class.



84

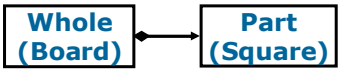
Association Classes

- Guideline: Clues that an association class might be useful in a domain model
  - An attribute is related to an association
  - Instances of the association class have a lifetime dependency on the association.
  - There is a many-to-many association between two concepts and information associated with the association itself.



Aggregation and Composition

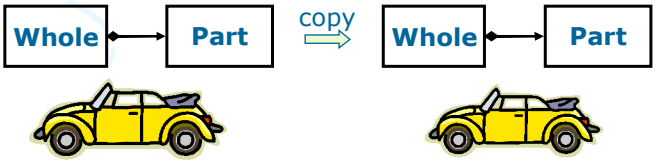
- Aggregation
  - is a vague kind of association in UML that loosely suggests whole-part relationship



- Composition
  - Is a strong kind of whole-part aggregation and is useful to show in some models
  - A instance of the part (e.g., Square) belongs to only one composite instance (e.g., Board) at a time
  - The part must always belong to a composite (no free-floating Fingers)
  - The composite is responsible for the creation and deletion of its parts
    - If the composite is destroyed, its parts must either be destroyed or attached to another composite
- Guideline: Don't bother to use aggregation in UML; rather, use composition when appropriate

Aggregation and Composition

- Guideline: On composition: if in doubt, leave it out.
- Guideline: Consider showing composition when:
  - The lifetime of the part is bound within the lifetime of the composite – there is a create-delete dependency of the part on the whole
  - There is an obvious whole-part physical or logical assembly
  - Some properties of the composite propagate to the parts, such as the location
  - Operations applied to the composite propagate to the parts, such as destruction, movement, recording



87

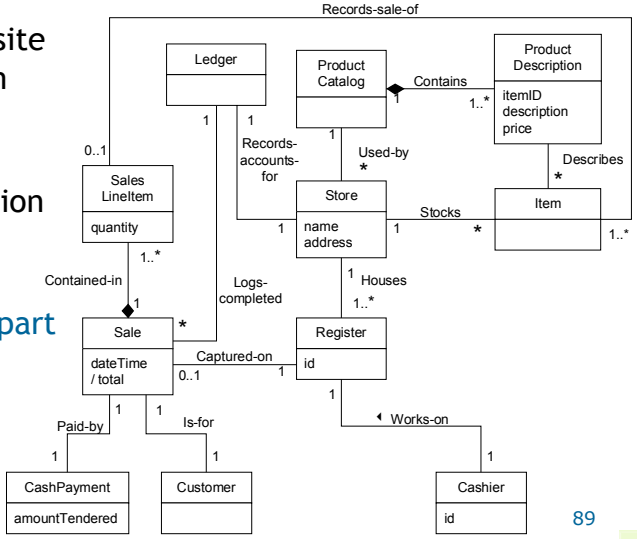
Aggregation and Composition

- Discover and show composition has the following benefits, most which relate to design rather than analysis
  - It clarifies the domain constraints regarding the eligible existence of the part independent of the whole. The part may not exist outside of the whole
    - During design work, this has an impact on the create-delete dependencies between the whole and part software classes and database elements (in terms if referential integrity and cascading delete paths)
  - It assists in the identification of a creator (the composite) using the GRASP pattern
  - Operations – such as copy and paste – applied to the whole often propagate to the parts

88

Aggregation and Composition

- Sale is a composite of SalesLineItem
- Product is a composite of ProductDescription
- No other relationship suggests whole-part semantics, and create-delete dependency
  - Leave it out

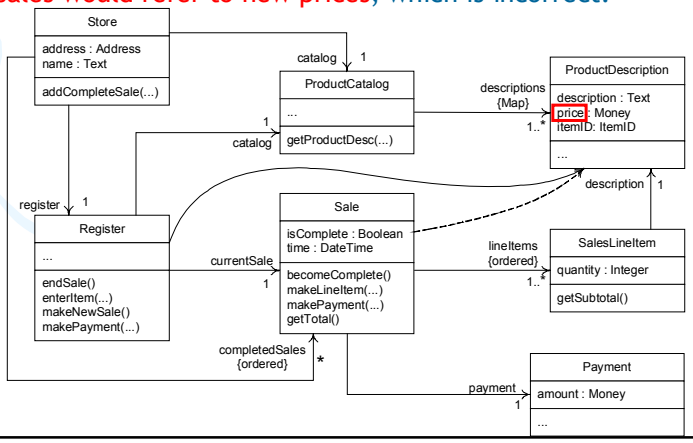


Iteration 1 domain model

89

Time Intervals and Product Prices – Fixing an Iteration 1 "Error"

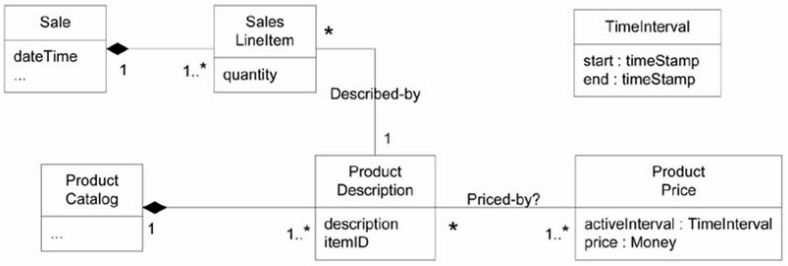
- Issue of time intervals associated with information …
  - If a SalesLineItem always retrieved the current price recorded in a ProductDescriptions, then when the price was changed, old sales would refer to new prices, which is incorrect.



90

Time Intervals and Product Prices – Fixing an Iteration 1 "Error"

- Two ways to model time intervals
  1. Copy the product price into SalesLineItem, and maintain the current price in the ProductDescriptions
  2. (More robust) Associate a collection of ProductPrices with a ProductDescriptions, each with an associated applicable time interval
     - The organization can record all past prices to resolve the sale price problem, and for trend analysis
     - Record future planed prices



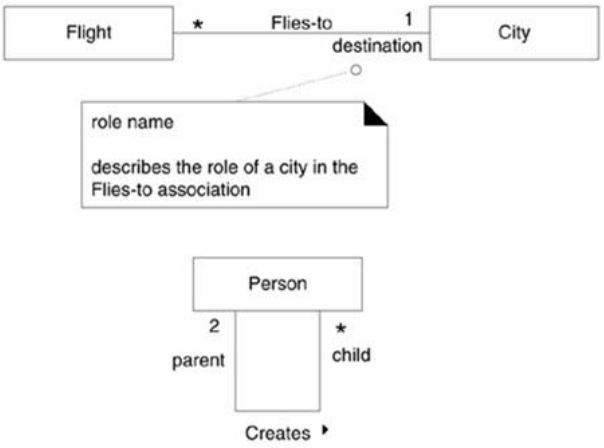
91

Association Role Names

- Each end of an association is a role with
  - Name (next page)
    - Describes the role played by objects in the association
    - An explicit role name is not required – it is useful when the role of the object is not clear
    - Usually starts with a lowercase letter
    - If not explicitly present, assume that the default role name is equal to the related class name, though starting with a lowercase letter
    - Roles used in DCDs may be interpreted as the basis for attribute names during code generation
  - Multiplicity

92

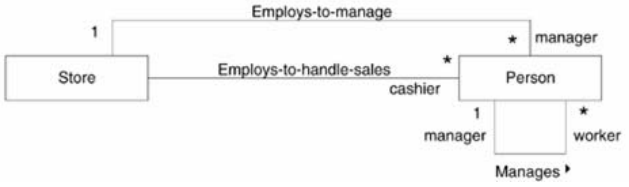
Association Role Names



93

Roles as Concepts versus Roles in Associations

- Roles in associations
  - A relatively accurate way to express the notion that the same person takes on multiple roles



- Roles as concepts
  - Provide ease and flexibility in adding unique attributes, and additional semantics
  - Implementation of roles as separate classes is easier – it is not convenient to dynamically mutate an instance of one class into another

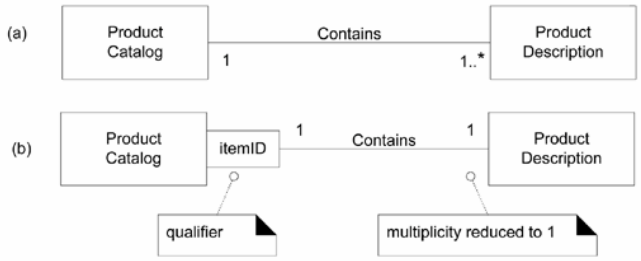
94

Derived Elements

- Derived elements
  - Attributes and associations are the most common derived elements
- Guideline
  - Avoid showing derived elements in a diagram, since they add complexity without new information.
  - However, add a derived element when it is prominent in the terminology, and excluding it impairs comprehension

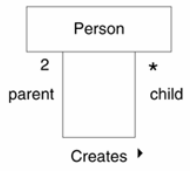
Qualified Associations

- An association with a qualifier is a qualified association
  - A qualifier distinguishes the set of objects at the far end of the association based on the qualifier value
    - E.g., ProductDescriptions may be distinguished in a ProductCatalog by their itemID
  - Depicting a qualifier in a domain model communicates how the things of one class are distinguished in relation to another class



Reflexive Associations

- Reflexive association
  - A concept may have an association to itself



97

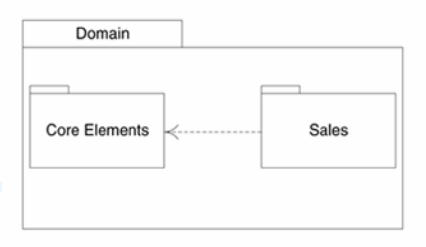
Using Packages to Organize Domain Model

- A domain model can easily grow large enough that it is desirable to factor it into packages of strongly related concepts
- UML package ownership and references
  - An element is owned by the package with in which it is defined, but may be referenced in other packages
  - A class shown in a foreign package may be modified with new associations, but must otherwise remain unchanged

98

Using Packages to Organize Domain Model

- Package dependencies
  - If a model element is in some way dependent on another, the dependency may shown with a dependency relationship (arrowed line)
  - E.g., Sales packages has a dependency on the Core Elements package



99

Using Packages to Organize Domain Model

- How to partition the Domain Model?
- Guideline: To partition the domain model into packages, place elements together that:
  - are in the same subject area – closely related by concept or purpose
  - are in a class hierarchy together
  - participate in the same use cases
  - are strongly associated
- It is useful if
  - All elements related to the domain model are rooted in a package called Domain,
  - All widely shared, common, core concepts are defined in a package named Core Elements or Common Concepts

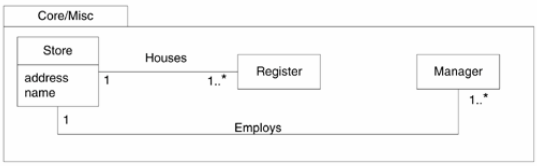
100

Using Packages to Organize Domain Model

Iteration 1 domain model

- POS Domain Model Packages

- Core/Misc Package
  - Is useful to own widely shared concepts or those without an obvious home



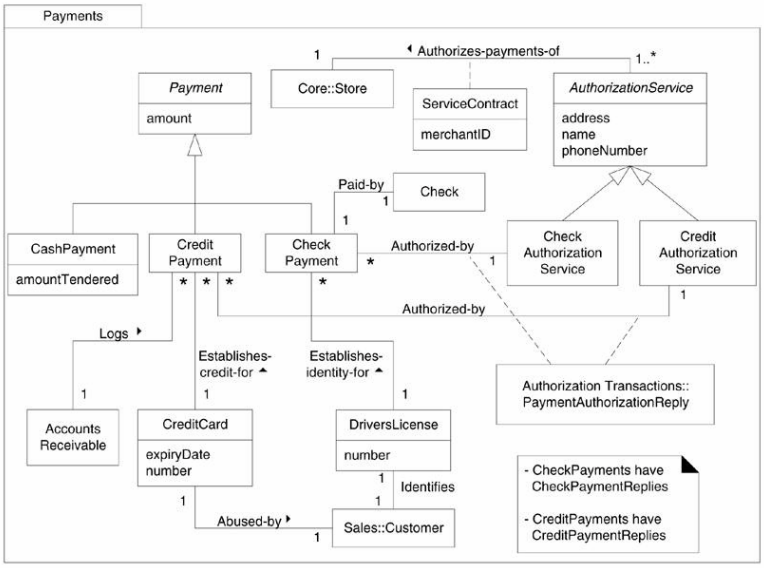
101

Using Packages to Organize Domain Model

- Payments package (next page)
  - As in iteration 1, new associations are primarily motivated by a need-to-know criterion,
    - E.g., there is a need to remember the relationship between CreditPayment and CreditCard
  - Some associations are added for comprehension
    - E.g., DriversLicense Identifies Customer
  - Note that PaymentAuthorizationReply is expressed as an association class - a reply arises out of association between a payment and its authorization service

102

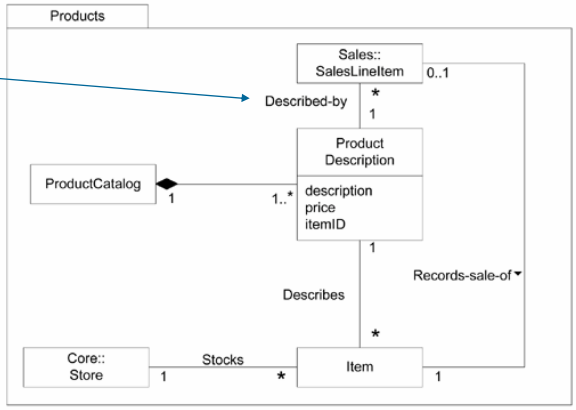
Using Packages to Organize Domain Model



Using Packages to Organize Domain Model

- Products package
  - No new concepts, except for composite aggregation

Not shown in the previous iterations

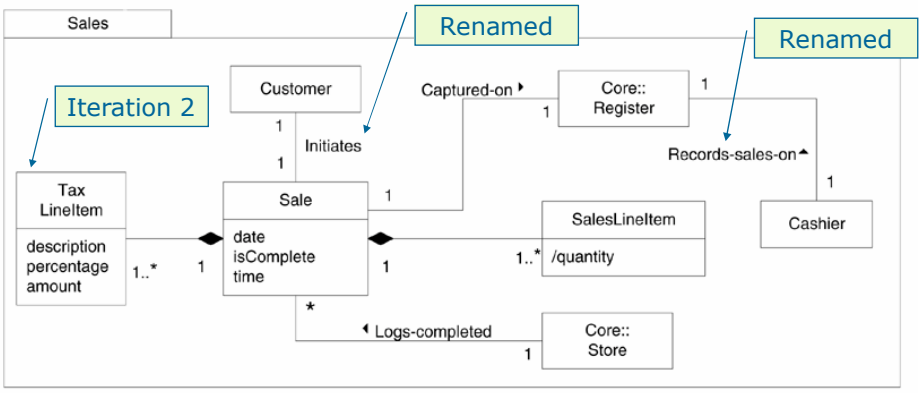


Iteration 1 domain model

Using Packages to Organize Domain Model

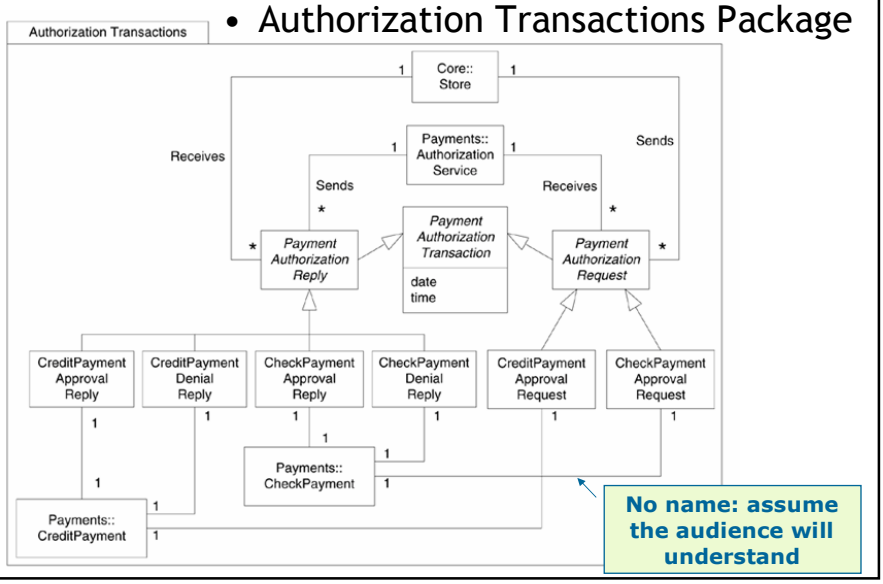
Iteration 1 domain model

- Sales package
  - No new concepts, except for composite aggregation and derived attributes



Using Packages to Organize Domain Model

- Authorization Transactions Package



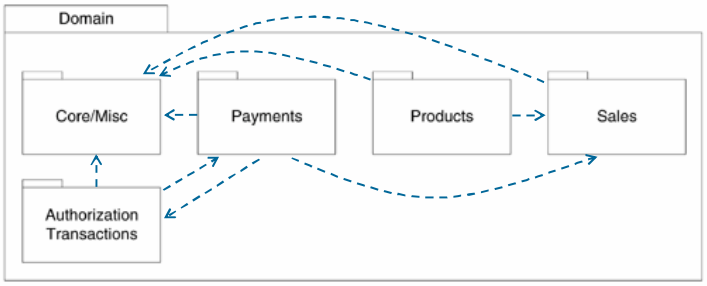
Using Packages to Organize Domain Model

- Monopoly Domain model Refinements (next page)
  - Different kinds of property squares (LotSquare …)
    - If the domain rules treat a noteworthy concept in a different or distinct manner, then show it a separate specialization
  - An abstract superclass PropertySquare
    - All the subclasses have a price attribute and an Owns association with a player

107

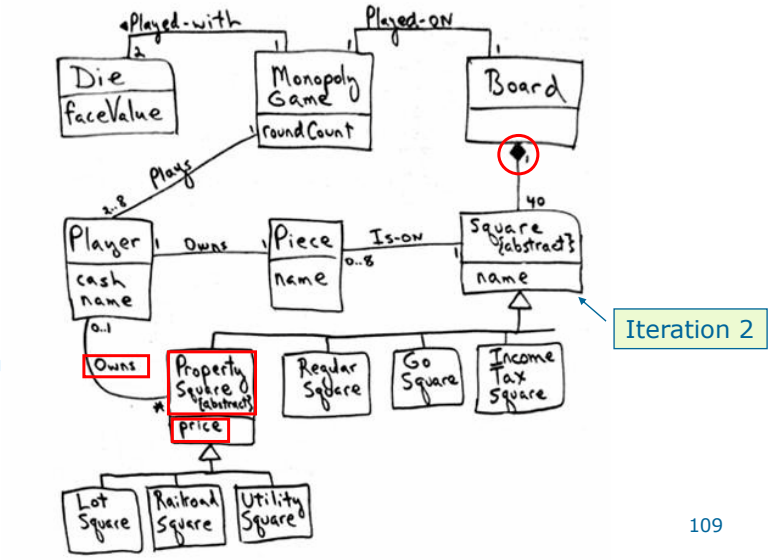
Using Packages to Organize Domain Model

- Dependency in POS Domain Model Packages



108

Using Packages to Organize Domain Model



Conclusion

- Iterative development
- Requirements / Use cases
- Embrace change
- OOAD
- GRASP
- UML
- Reuse
- Unit test
- Team work / communication